

Foundations of Cybersecurity

14 June 2022

Exercise n. 1 [10]

State and prove the Shannon's Theorem.

Exercise n. 2 [10]

The hash function $H(\cdot)$ suffers from both collision attacks in 2^{41} , exploiting the inner structure function and simply appending properly crafted blocks to the message. Besides this, the size of $H(\cdot)$, 128 bits, limits its maximum collision resistance to 2^{64} .

Willing to strengthen the $H(\cdot)$, three proposals are made:

- a) $H-A(m)$ is defined as $SHA2-256(H(m))$
- b) $H-B(m)$ is defined as $H(SHA2-256(m))$
- c) $H-C(m)$ is defined as $AES_k(H(m))$, with a fixed key k
- d) $H-D(m)$ is defined as $SHA2-256(H(SHA2-256(m)))$

Comment on the improvements to the collision resistance of the resulting hash function.

Exercise n. 3 [10]

Find and explain the vulnerabilities of the following function. Then patch them.

```
void ExpandVector(std::vector<int>& c) {  
    // This code wants to expand the vector c by doubling each of its  
    // elements. For example, the vector [1,2,3] must become  
    // [1,1,2,2,3,3].  
    auto i = c.begin();  
    while (i != c.end()) {  
        c.insert(i, *i);  
        i++;  
    }  
}
```

Solution

Exercise n.1

See theory.

Exercise n.2

- a) H-A does not improve the collision resistance of plain H at all, as a message pair colliding in H will also collide in H-A, since SHA2-256 is a deterministic algorithm. It is thus possible to have collisions for H-A in 2^{41} steps, appending well-crafted block padding to any message.
- b) H-B prevents the attacker from exploiting the collisions relying on the inner structure of H. This is due to the computational infeasibility of controlling the output of the SHA2-256 hash function which is the input of H (note that controlling the output of SHA2-256 implies that SHA2-256 is not first-preimage attack robust). Assume that we can find two values z_1 and z_2 , such that, $H(z_1) = H(z_2)$. However, since SHA-256 is preimage resistance, it is not possible to find the respective preimages. H-B leaves still open the possibility of obtaining a collision in 2^{64} as the output of the H-B digest is 128 bits wide.
- c) H-C does not provide any improvements on the collision resistance front just like H-A, as having a deterministic bijective primitive processing the output of the original MD5 will still result in a collision in H being propagated to the output.
- d) H-D basically feeds the output of H-B into SHA2-256. Thus, as H-B has a collision resistance of 2^{128} , and equal inputs to SHA2-256 will yield equal outputs also H-D has a collision resistance of 2^{128} .

Exercise n.3

There are two problems in this code:

- a) it does not work because the function insert will try to insert a copy of the first element until the iterator i is invalidated. This is since the iterator “ i ” must be incremented twice. If the iterator is only incremented once, then the iterator “ i ” will point to the element just inserted.
- b) The insert may cause reallocation, therefore invalidating the iterator i . *“Causes reallocation if the new size() is greater than the old capacity(). If the new size() is greater than capacity(), all iterators and references are invalidated. Otherwise, only the iterators and references before the insertion point remain valid. The past-the-end iterator is also invalidated.”*¹

Therefore, the solution is quite simple, to assign the return of the insert function to the iterator “ i ” and then increment once more the iterator “ i ”:

```
void ExpandVector(std::vector<int>& c) {
    auto i = c.begin();
    while(i != c.end()) {
        i = c.insert(i, *i);
        i++;
        i++;
    }
}
```

¹ Sources: <https://en.cppreference.com/w/cpp/container/vector/insert>